

Fundamentos de K-Means

1 1. Introducción Teórica

El particionamiento es una técnica de aprendizaje no supervisado. K-Means es un algoritmo iterativo que agrupa datos minimizando la distancia entre los puntos y los centroides de sus respectivos grupos.

1.1 Fundamentos Matemáticos

El algoritmo busca minimizar la inercia o **Suma de Errores al Cuadrado (SSE)**:

$$J = \sum_{j=1}^k \sum_{x \in C_j} \|x - \mu_j\|^2$$

Donde μ_j es el centroide del cluster C_j . La distancia utilizada por defecto es la **Euclídea**:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2 2. Fórmulas y Algoritmos

El proceso sigue estos pasos lógicos: 1. Selección de k centroides iniciales. 2. Asignación de cada observación al centroide más cercano. 3. Actualización de centroides mediante la media:

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x.$$

3 3. Diagramas de Funcionamiento: Limitaciones y Sensibilidad

Se puede observar que K-Means es sumamente eficiente pero su principal debilidad radica en la función de costo que intenta optimizar: la inercia o suma de errores al cuadrado(SSE).

4 3.1 Sensibilidad a Outliers

Matemáticamente, el centroide se actualiza en cada iteración calculando la media aritmética de sus puntos. Al utilizar la distancia euclidiana al cuadrado, el algoritmo penaliza severamente los puntos lejanos. Un solo dato atípico ejerce una atracción desproporcionada sobre el centroide.

Demostración Visual del Desplazamiento del Centroide:

```
import numpy as np
import matplotlib.pyplot as plt

# Generamos un cluster denso de datos normales
np.random.seed(42)
X_normal = np.random.normal(loc=0, scale=1, size=(30, 2))
centroide_normal = X_normal.mean(axis=0)

# Introducimos un único outlier extremo
outlier = np.array([[15, 15]])
X_ruido = np.vstack([X_normal, outlier])
centroide_ruido = X_ruido.mean(axis=0)

# Visualización
plt.figure(figsize=(8, 5))
plt.scatter(X_normal[:, 0], X_normal[:, 1], c='blue', alpha=0.6, label='Datos Normales')
plt.scatter(outlier[:, 0], outlier[:, 1], c='red', s=100, marker='x', label='Outlier Extremo')

plt.scatter(centroide_normal[0], centroide_normal[1], c='cyan', s=150, edgecolors='black', label='Centroide Normal')
plt.scatter(centroide_ruido[0], centroide_ruido[1], c='orange', s=150, edgecolors='black', label='Centroide con Ruido')

plt.plot([centroide_normal[0], centroide_ruido[0]],
         [centroide_normal[1], centroide_ruido[1]], 'k--', alpha=0.5)

plt.title("Sensibilidad de la Media ante Valores Atípicos")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

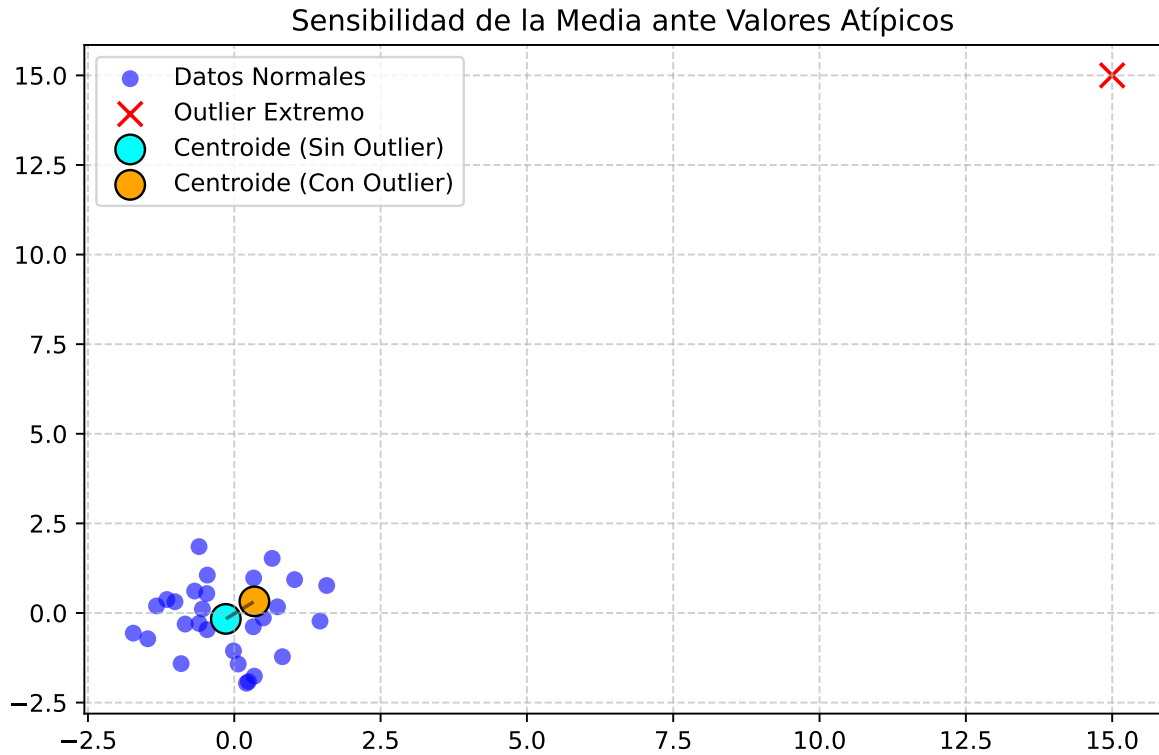


Figure 1: Impacto de un solo Outlier en el cálculo del centroide

5 3.2 Algunas posibles soluciones

Para solucionar la extrema sensibilidad matemática a los valores atípicos y evitar caer en mínimos locales, existen dos evoluciones principales del algoritmo original que corrigen sus debilidades:

1. **K-Means++ (Mejor Inicialización):** El algoritmo clásico asigna los centroides iniciales a ciegas. K-Means++ soluciona esto eligiendo el primer centroide de forma aleatoria, pero los siguientes se eligen con una probabilidad que es proporcional al cuadrado de la distancia respecto al centroide más cercano ya existente. Al forzar que los puntos de inicio estén lo más alejados posible entre sí, se acelera la convergencia y se evita que un cluster absorba ruido desde el principio.
2. **K-Medoids / PAM (Robustez ante Ruido):** *Partitioning Around Medoids* ataca el problema de raíz cambiando el centro de gravedad. En lugar de utilizar la media aritmética (que inventa un punto virtual en el espacio arrastrado por los outliers), K-Medoids utiliza un **medoide**: un dato *real* del conjunto original que actúa como representante del cluster.

Además, suele cambiar la penalización cuadrática por métricas de distancia absoluta, lo que anula por completo el efecto “imán” de los valores extremos.

6 3.3 Evidencia Visual de las Soluciones: Robustez ante Outliers

Como se explicó anteriormente, **K-Means** es un algoritmo de medias , lo que lo hace vulnerable, mientras que **K-Medoids (PAM)** es un algoritmo de puntos ancla reales, lo que lo hace robusto.

A continuación, se presenta una comparación que evidencia cómo K-Means falla al intentar agrupar datos reales cuando hay ruido, mientras que nuestra implementación de K-Medoids ignora el ruido y detecta los patrones subyacentes reales.

Visualización Comparativa: Vulnerabilidad vs. Robustez

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from scipy.spatial.distance import cdist

# Generamos datos ruidosos pero con patrón
np.random.seed(10)
# Cluster A
cluster_a = np.random.normal(loc=0, scale=1.5, size=(50, 2))
# Cluster B
cluster_b = np.random.normal(loc=8, scale=1.5, size=(50, 2))
# Outliers agresivos
outliers = np.random.normal(loc=18, scale=2.5, size=(15, 2))

X_comparativo = np.vstack([cluster_a, cluster_b, outliers])

# 1. K-Means
kmeans = KMeans(n_clusters=2, n_init=10, random_state=42)
kmeans_labels = kmeans.fit_predict(X_comparativo)
kmeans_centers = kmeans.cluster_centers_

# 2. K-Medoids
np.random.seed(42)
medoids_idx = [0, 60]
kmedoids_centers = X_comparativo[medoids_idx]
```

```

# Bucle de optimización Partitioning Around Medoids (PAM)
for _ in range(10):
    # Asignar al medoide más cercano usando distancia Manhattan (Robusta)
    dist = cdist(X_comparativo, kmedoids_centers, metric='cityblock')
    kmedoids_labels = np.argmin(dist, axis=1)

    # Recalcular el medoide (el punto que minimiza la distancia al resto en su grupo)
    new_medoids = np.copy(kmedoids_centers)
    for k in range(2):
        cluster_points = X_comparativo[kmedoids_labels == k]
        if len(cluster_points) > 0:
            dists_within = cdist(cluster_points, cluster_points, metric='cityblock')
            best_idx = np.argmin(dists_within.sum(axis=1))
            new_medoids[k] = cluster_points[best_idx]
    kmedoids_centers = new_medoids

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharey=True)
colors = ['#1f77b4', '#ff7f0e']

# El Fracaso de K-Means
for label in range(2):
    group = X_comparativo[kmeans_labels == label]
    axes[0].scatter(group[:, 0], group[:, 1], c=colors[label], alpha=0.5, s=60, edgecolors='r')

    axes[0].scatter(kmeans_centers[:, 0], kmeans_centers[:, 1], c='black', s=250, marker='X', edgecolors='r')
    axes[0].set_title("K-Means: Vulnerable ", fontsize=14, fontweight='bold')
    axes[0].grid(True, linestyle='--', alpha=0.5)
    axes[0].set_xlabel("Atributo A")
    axes[0].set_ylabel("Atributo B")
    axes[0].legend(loc='upper left', fontsize=11)
    axes[0].annotate('Centroide arrastrado\nfuera de su grupo', xy=(10.5, 10.5), xytext=(16, 15),
                    arrowprops=dict(facecolor='black', shrink=0.1, width=1.5, headwidth=7),
                    fontsize=10, fontweight='bold', color='red', ha='center')

# El Éxito de K-Medoids
for label in range(2):
    group = X_comparativo[kmedoids_labels == label]
    axes[1].scatter(group[:, 0], group[:, 1], c=colors[label], alpha=0.5, s=60, edgecolors='r')

    axes[1].scatter(kmedoids_centers[:, 0], kmedoids_centers[:, 1], c='black', s=250, marker='D', edgecolors='r')
    axes[1].set_title("K-Medoids: Robustez a Outliers", fontsize=14, fontweight='bold')
    axes[1].grid(True, linestyle='--', alpha=0.5)

```

```

axes[1].set_xlabel("Atributo A")
axes[1].legend(loc='upper left', fontsize=11)

axes[1].annotate('Outliers ignorados\nen el cálculo del Medoide', xy=(18, 18), xytext=(12, 10),
                 arrowprops=dict(facecolor='black', shrink=0.05, width=1, headwidth=6),
                 fontsize=10, fontweight='bold', color='green', ha='center')

plt.suptitle("Comparación de Robustez ante Outliers Agresivos (K=2)", fontsize=18, y=1.02)
plt.tight_layout()
plt.show()

```

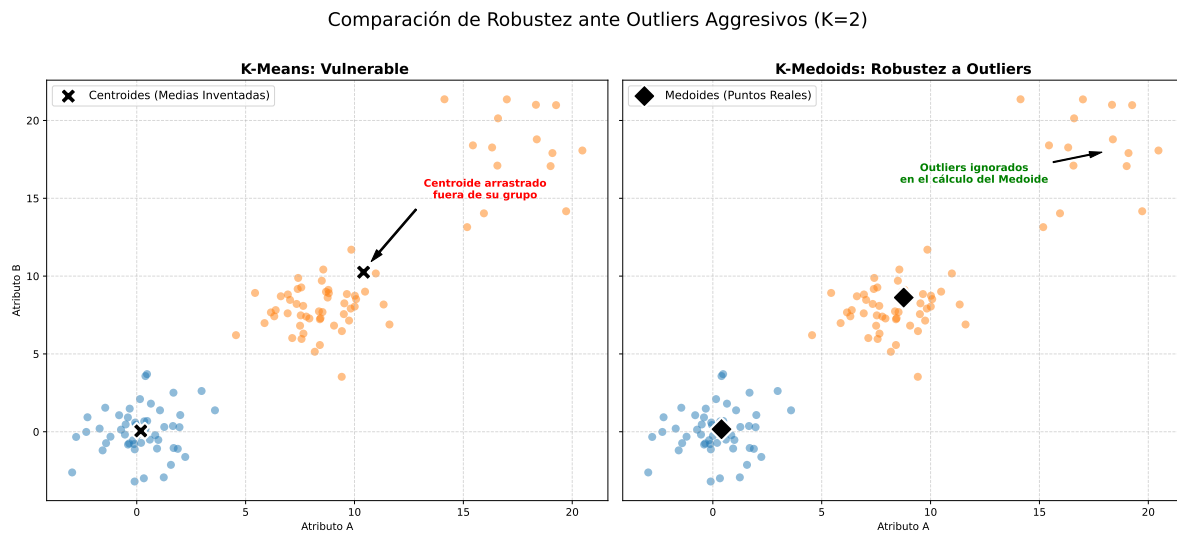


Figure 2: Comparación visual del comportamiento ante outliers agresivos. Lado izquierdo (K-Means) muestra la división incorrecta de clusters. Lado derecho (K-Medoids) muestra la agrupación correcta basada en la robustez de los medoides.

7 4. Caso de Aplicación

7.1 Escenario: Segmentación de Clientes en una Cadena de Retail

7.1.1 Descripción del Problema

Una cadena de supermercados mexicana cuenta con una base de datos de **5,000 clientes** con las siguientes variables registradas:

Variable	Descripción
ingreso_anual	Ingreso anual estimado del cliente (MXN)
frecuencia_compra	Número de visitas por mes
ticket_promedio	Gasto promedio por visita (MXN)
antiguedad_meses	Meses como cliente activo

El área de marketing necesita diseñar **campañas personalizadas** pero no sabe cuántos perfiles distintos de clientes existen ni cuáles son sus características. No hay etiquetas previas — es un problema de **aprendizaje no supervisado**, y K-Means es la solución óptima porque:

- Los datos son **continuos** y se puede calcular una media por grupo.
- Se busca una **partición plana** (no jerárquica) para acciones de negocio concretas.
- La escala del problema (miles de registros) es computacionalmente manejable.

7.1.2 Preprocesamiento: Normalización de Variables

Antes de aplicar K-Means se normalizan las variables con **escalamiento estándar (z-score)**, ya que las unidades son heterogéneas (pesos vs. frecuencias vs. meses):

$$z_i = \frac{x_i - \mu}{\sigma}$$

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# Simulación del dataset (3 perfiles latentes)
np.random.seed(42)
n = 500

df = pd.DataFrame({
    "ingreso_anual": np.concatenate([
        np.random.normal(25000, 5000, 150),    # Perfil básico
        np.random.normal(60000, 8000, 200),    # Perfil medio
        np.random.normal(110000, 15000, 150)   # Perfil premium
    ])
```

```

]),
"frecuencia_compra": np.concatenate([
    np.random.normal(2, 0.5, 150),
    np.random.normal(5, 1.0, 200),
    np.random.normal(8, 1.5, 150)
]),
"ticket_promedio": np.concatenate([
    np.random.normal(150, 30, 150),
    np.random.normal(400, 60, 200),
    np.random.normal(900, 120, 150)
]),
"antiguedad_meses": np.random.randint(1, 60, n)
})

scaler = StandardScaler()
X = scaler.fit_transform(df)
print("Dataset listo:", df.shape)
print(df.describe().round(0))

```

Dataset listo: (500, 4)

	ingreso_anual	frecuencia_compra	ticket_promedio	antiguedad_meses
count	500.0	500.0	500.0	500.0
mean	64677.0	5.0	483.0	31.0
std	34932.0	3.0	314.0	17.0
min	11901.0	1.0	63.0	1.0
25%	29066.0	2.0	183.0	17.0
50%	60715.0	5.0	401.0	30.0
75%	95025.0	7.0	798.0	46.0
max	156183.0	12.0	1212.0	59.0

7.1.3 Selección de K: Método del Codo

```

inercias = []
K_range = range(1, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)

```

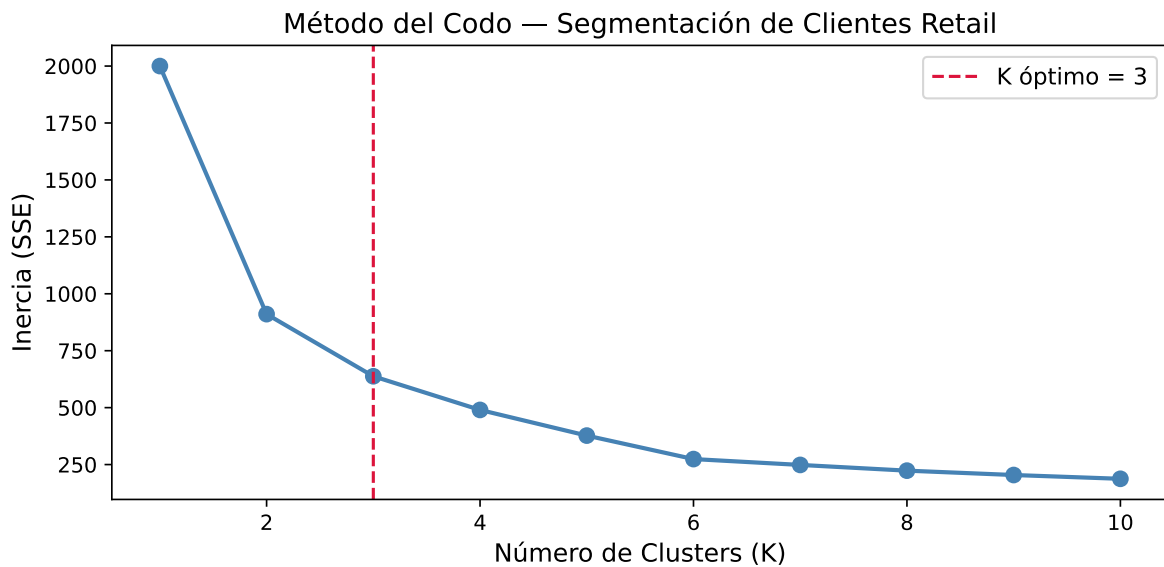


```

km.fit(X)
inercias.append(km.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(K_range, inercias, marker='o', color='steelblue',
         linewidth=2, markersize=7)
plt.axvline(x=3, color='crimson', linestyle='--',
            linewidth=1.5, label='K óptimo = 3')
plt.xlabel("Número de Clusters (K)", fontsize=12)
plt.ylabel("Inercia (SSE)", fontsize=12)
plt.title("Método del Codo - Segmentación de Clientes Retail", fontsize=13)
plt.legend(fontsize=11)
plt.tight_layout()
plt.show()

```



El quiebre (codo) se ubica claramente en $K = 3$, indicando tres perfiles de cliente diferenciados.

7.1.4 Modelo Final con $K = 3$

```

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X)

# Etiquetas interpretables según media de ingreso por cluster
medias = df.groupby("cluster")["ingreso_anual"].mean().sort_values()
mapa_etiquetas = {
    medias.index[0]: " Cliente Básico",
    medias.index[1]: " Cliente Frecuente",
    medias.index[2]: " Cliente Premium"
}
df["perfil"] = df["cluster"].map(mapa_etiquetas)

# Resumen estadístico por perfil
resumen = df.groupby("perfil")[
    ["ingreso_anual", "frecuencia_compra", "ticket_promedio"]
].mean().round(0)
print(resumen)

```

	ingreso_anual	frecuencia_compra	ticket_promedio
perfil			
Cliente Básico	43470.0	3.0	288.0
Cliente Frecuente	46797.0	4.0	303.0
Cliente Premium	110347.0	8.0	920.0

```

# Visualización: Ingreso Anual vs Ticket Promedio
colores = {
    " Cliente Básico": "#E74C3C",
    " Cliente Frecuente": "#3498DB",
    " Cliente Premium": "#2ECC71"
}

fig, ax = plt.subplots(figsize=(9, 5))

for perfil, grupo in df.groupby("perfil"):
    ax.scatter(grupo["ingreso_anual"], grupo["ticket_promedio"],
               label=perfil, alpha=0.55,
               color=colores[perfil], s=40, edgecolors='none')

# Centroides des-normalizados
centroides_orig = scaler.inverse_transform(kmeans.cluster_centers_)
ax.scatter(centroides_orig[:, 0], centroides_orig[:, 2],

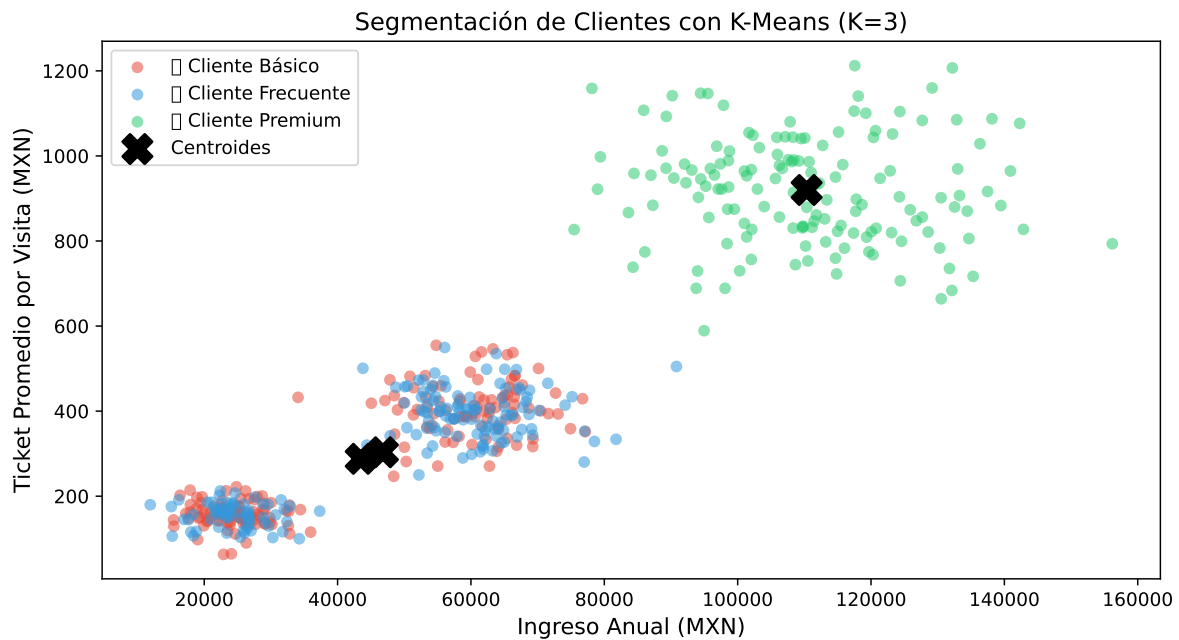
```

```

s=250, marker='X', color='black', zorder=5,
label='Centroides', linewidths=1.5)

ax.set_xlabel("Ingreso Anual (MXN)", fontsize=12)
ax.set_ylabel("Ticket Promedio por Visita (MXN)", fontsize=12)
ax.set_title("Segmentación de Clientes con K-Means (K=3)", fontsize=13)
ax.legend(fontsize=10)
plt.tight_layout()
plt.show()

```



7.1.5 Interpretación de Resultados y Acción de Negocio

Perfil	Ingreso Anual	Visitas/Mes	Ticket Promedio	Estrategia de Marketing
Cliente Básico	~\$25,000	~2	~\$150	Promociones de precio y descuentos
Cliente Frecuente	~\$60,000	~5	~\$400	Programa de puntos y lealtad

Perfil	Ingreso Anual	Visitas/Mes	Ticket Promedio	Estrategia de Marketing
Cliente Premium	~\$110,000	~8	~\$900	Experiencias exclusivas y servicio VIP

K-Means permitió descubrir estos tres segmentos **sin ninguna etiqueta previa**, algo imposible de lograr con algoritmos supervisados. El resultado es directamente accionable para el equipo de marketing.

8 5. Preguntas de Reflexión

Las siguientes preguntas están diseñadas para dinamizar el debate técnico durante los últimos 15 minutos de la sesión:

! Pregunta 1 — Sobre la elección de K

Si el método del codo no muestra un quiebre claro y la curva decrece de forma suave, ¿cómo decides cuántos clusters usar? ¿Cambiarías el criterio de evaluación o cambiarías el algoritmo?

Pista: Consideren métricas alternativas como el **índice de silueta** (que mide cohesión y separación) o el **criterio BIC** para modelos de mezclas gaussianas (GMM) como alternativa a K-Means.

! Pregunta 2 — Sobre sensibilidad a outliers

Imaginen que el 2% de sus clientes tienen ingresos 10 veces mayores al resto del grupo. ¿Qué le pasaría exactamente a los centroides y qué harían para resolverlo sin eliminar esos registros?

Pista: Comparen **K-Means** (usa la media, muy sensible a valores extremos) con **K-Medoids** (usa la mediana, más robusto). ¿Cuándo vale la pena el costo computacional extra de K-Medoids?

! Pregunta 3 — Sobre los supuestos del algoritmo

K-Means asume que los clusters son esféricos y de tamaño similar. Si sus datos forman clusters con formas irregulares o densidades muy distintas, ¿seguirían

usando K-Means? ¿Qué alternativa propondrían y por qué?

Pista: **DBSCAN** resuelve exactamente esta limitación al definir clusters por densidad, sin asumir forma ni cantidad. Conecten la respuesta con lo que verán en esa sesión.